

Multivariate Models

Set up

```
library(fpp3)
library(tidyverse)
library/slider)
library(gridExtra)
```

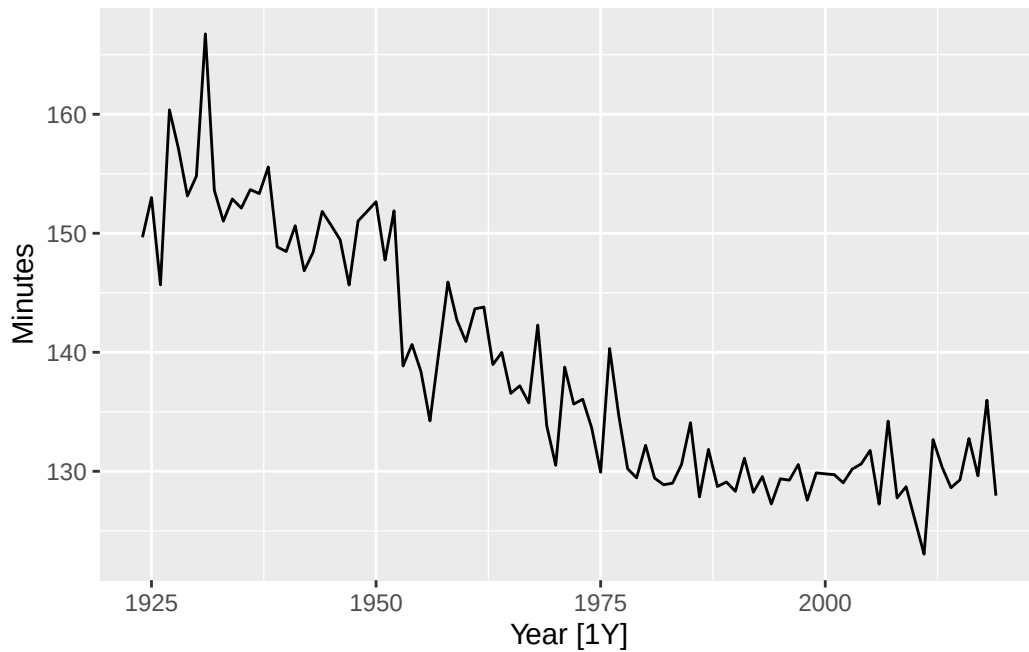
1. Nonlinear trend

In this problem, we will analyze the `boston_marathon` dataset, which records the winning times for the race from 1897 to 2019 (across several categories). We will focus on the men's event from 1924 onwards. Use the following code snippet to extract the relevant time series.

```
boston_men <- boston_marathon |>
  filter(Year >= 1924) |>
  filter(Event == "Men's open division") |>
  mutate(Minutes = as.numeric(Time)/60)
```

- Make a time plot of the time series.
- If you were fitting a piecewise linear trend, where would you place the knots?
- Fit three models: (i) A linear trend model, (ii) An exponential trend model, (iii) A piecewise linear trend model with the knots selected in b).
- Plot the forecasts of all three methods and comment.
- Compare the AICc values of the models.
- Comment on why it may or may not be fair to select from these models using AICc.

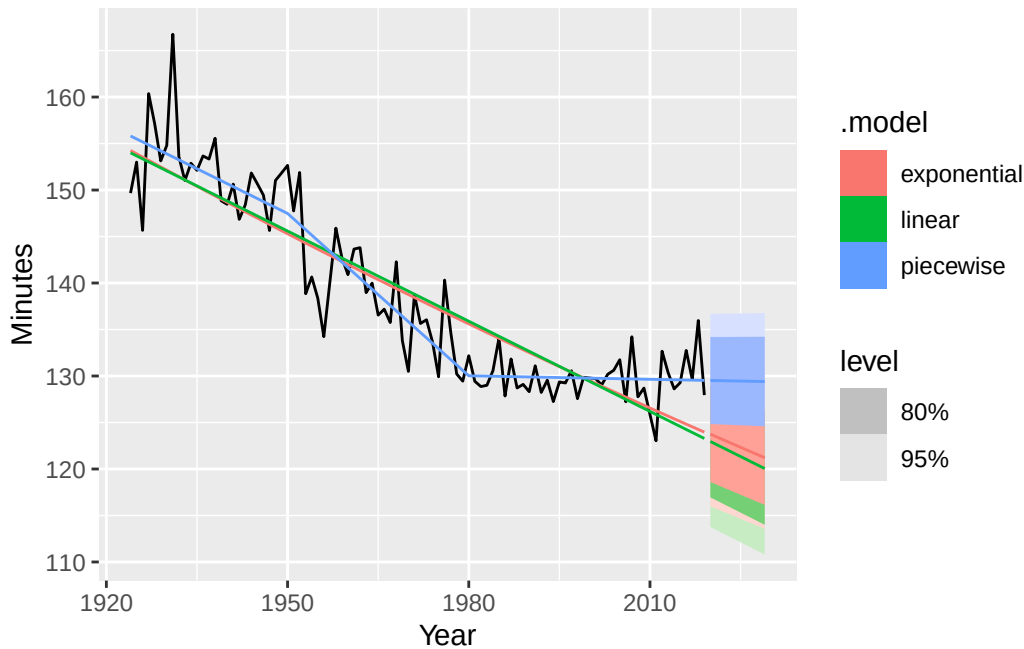
```
boston_men |>
  autoplot(Minutes)
```



The knots should be placed where the slope changes. In this case, at 1950 and 1980.

```
boston_fit <- boston_men |>
  model(
    linear = TSLM(Minutes ~ trend()),
    exponential = TSLM(log(Minutes) ~ trend()),
    piecewise = TSLM(Minutes ~ trend(knots = c(1950, 1980)))
  )

boston_fit |>
  forecast(h = 10) |>
  autoplot(boston_men) +
  geom_line(data = augment(boston_fit),
    aes(x = Year, y = .fitted, color = .model))
```



The piecewise linear trend seems to have the best fit. The forecasts for the linear and exponential trend methods seem to be biased downwards.

```
boston_fit |>
  glance()
```

```
# A tibble: 3 x 16
  Event .model r_squared adj_r_squared sigma2 statistic p_value df log_lik
<fct> <chr>   <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <int> <dbl>
1 Men's~ linear    0.795      0.793 2.11e+1    365. 3.85e-34    2 -282.
2 Men's~ expon~    0.802      0.800 1.03e-3    380. 8.48e-35    2  195.
3 Men's~ piece~    0.884      0.880 1.22e+1    234. 6.83e-43    4 -254.
# i 7 more variables: AIC <dbl>, AICc <dbl>, BIC <dbl>, CV <dbl>,
#   deviance <dbl>, df.residual <int>, rank <int>
```

The exponential trend model has the smallest AICc value. However, because it has performed a transformation of the observations, the likelihood function of the model is entirely different, and it does not make sense to compare its AICc value with those of the other models.

The linear trend and piecewise linear trend models are indeed directly comparable. As expected, the piecewise linear trend model has a smaller AICc model. In general, however, we need to be careful. Because we selected the knots of the piecewise linear trend model *after* having seen the data. This is some additional degrees of freedom that was not accounted for and difficult to model. (Called researcher degrees of freedom).

2. Scenario-based forecasting

Consider the `us_change` dataset, which comprises percentage changes in quarterly personal consumption expenditure, personal disposable income, production, savings and the unemployment rate for the US from 1970 to 2016. Imagine you are a policy maker and would like to know what would happen to change in consumption under two different scenarios for change in income, savings, and unemployment.

- Fit a time series regression model of Consumption on Income, Savings, and Unemployment (we refer to the various time series by their column names). What are the fitted coefficients?
- We want to examine two scenarios for 2019 Q3 to 2020 Q2 (next 4 quarters). Under scenario 1, there will be a constant growth of 1% and 0.5% respectively for income and savings, with no change in unemployment. Under scenario 2, there will be a constant decrease of 1% and 0.5% respectively for income and savings, with no change in unemployment. Create tsibbles with these values using the `new_data()` function.
- Forecast change in consumption under the two scenarios in b). Plot and interpret your results.
- When performing the analysis in c), did we view the model in a) as a predictive model or as a causal model?

```
us_fit <- us_change |>
  model(
    lm = TSLM(Consumption ~ Income + Savings + Unemployment)
  )
us_fit |> tidy()
```

```
# A tibble: 4 x 6
```

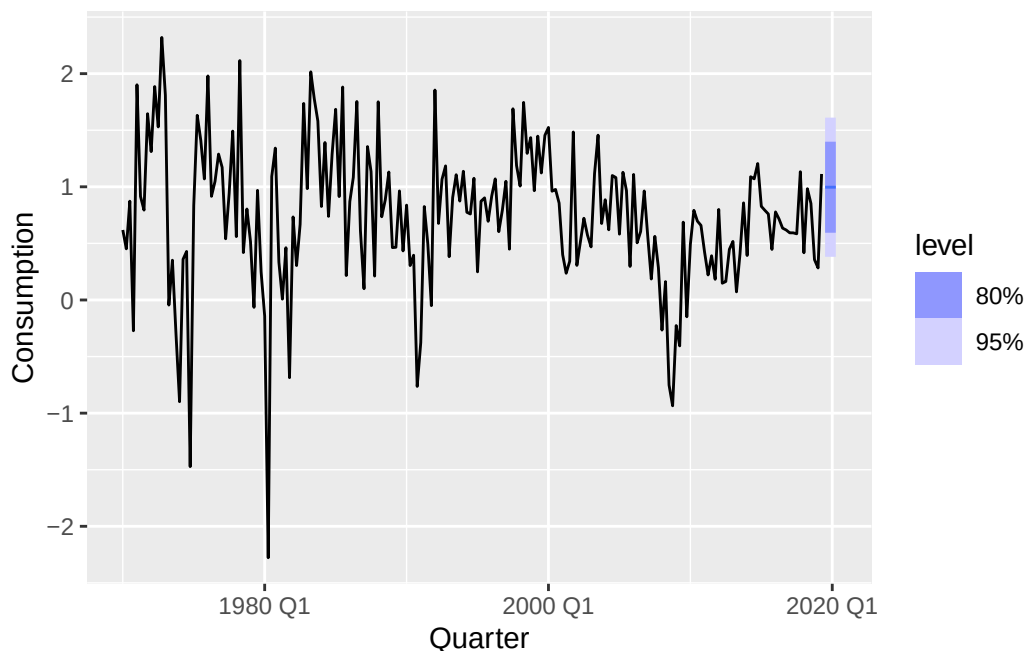
	.model	term	estimate	std.error	statistic	p.value
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	lm	(Intercept)	0.266	0.0341	7.81	3.57e-13
2	lm	Income	0.757	0.0396	19.1	1.91e-46
3	lm	Savings	-0.0537	0.00292	-18.4	2.55e-44
4	lm	Unemployment	-0.313	0.0678	-4.62	7.06e- 6

```
scenario1_dat <-
  new_data(us_change, 4) |>
  mutate(Income = 1, Savings = 0.5, Unemployment = 0)

scenario2_dat <-
  new_data(us_change, 4) |>
  mutate(Income = -1, Savings = -0.5, Unemployment = 0)
```

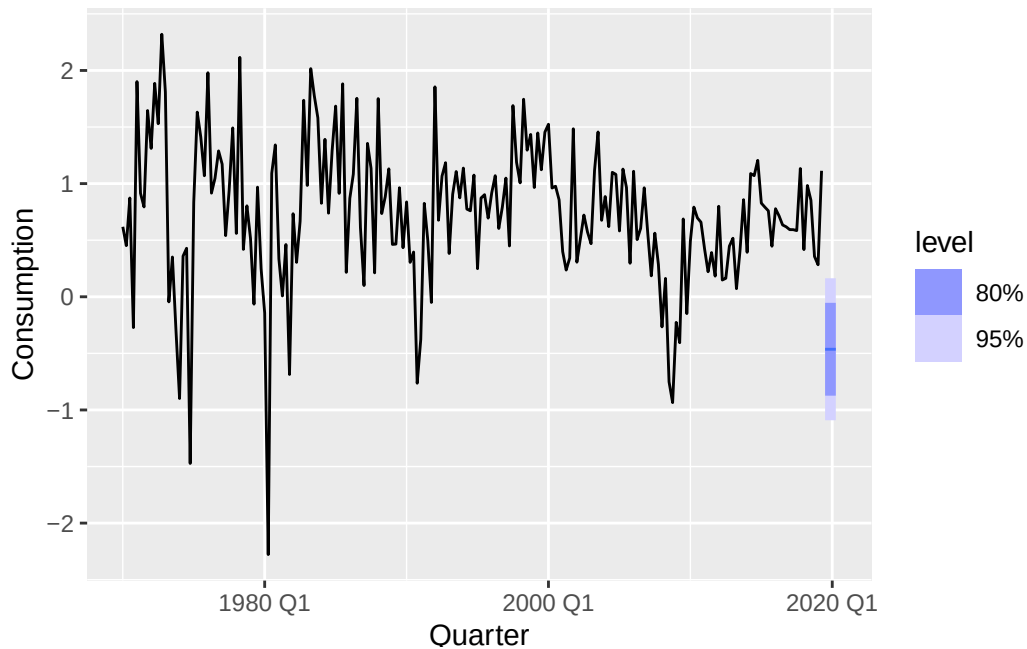
```
us_fit |>
  forecast(new_data = scenario1_dat, h = 4) |>
  autoplot(us_change)
```

Warning: Input forecast horizon `h` will be ignored as `new\_data` has been provided.



```
us_fit |>
  forecast(new_data = scenario2_dat, h = 4) |>
  autoplot(us_change)
```

Warning: Input forecast horizon `h` will be ignored as `new\_data` has been provided.



Under scenario 1, we forecast change in consumption to be about 0.996%. Under scenario 2, we forecast change in consumption to be about -0.464%.

We viewed the model as a causal model. These scenarios involved intervening to change variables via policy, and we are interested in the true value of change in consumption (policy outcome).

3. Multiple seasonality

Consider the `bank_calls` dataset, which records the number of calls to a North American commercial bank per 5-minute interval between 7:00am and 9:05pm each weekday over a 33 week period. For computational tractability, we restrict the time series to the first 4 weeks.

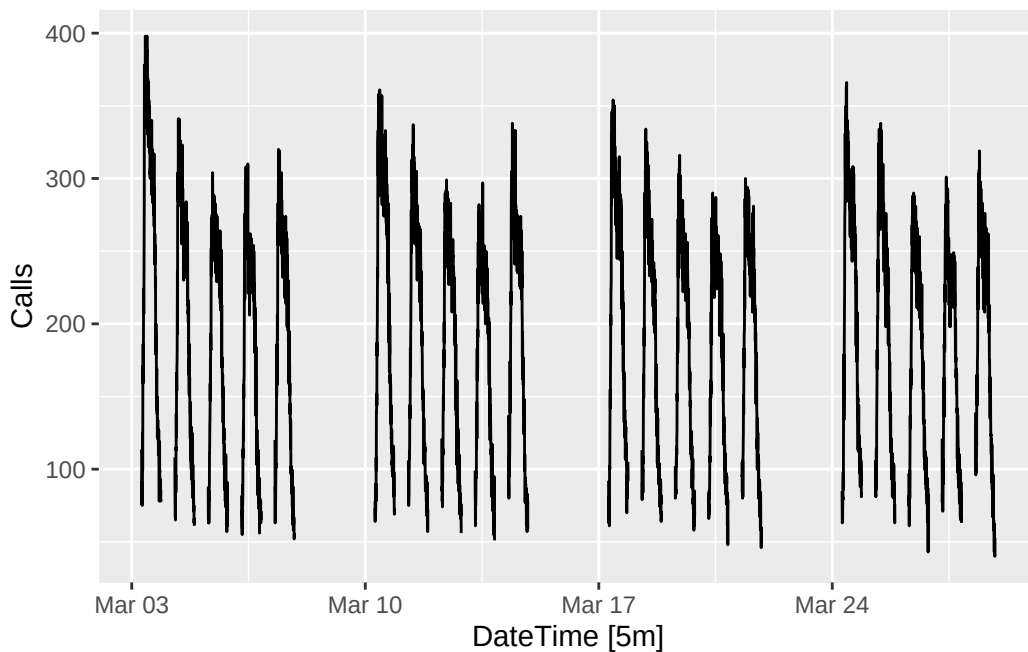
```
bank_calls_filtered <- bank_calls |>
  filter_index(~ "2003-03-30")
```

- Make a time plot. What types of seasonality do you observe?
- There are many gaps in the time series so we use the following snippet to index according to the observation number.

```
calls <- bank_calls_filtered |>
  mutate(t = row_number()) |>
  update_tsibble(index = t, regular = TRUE)
```

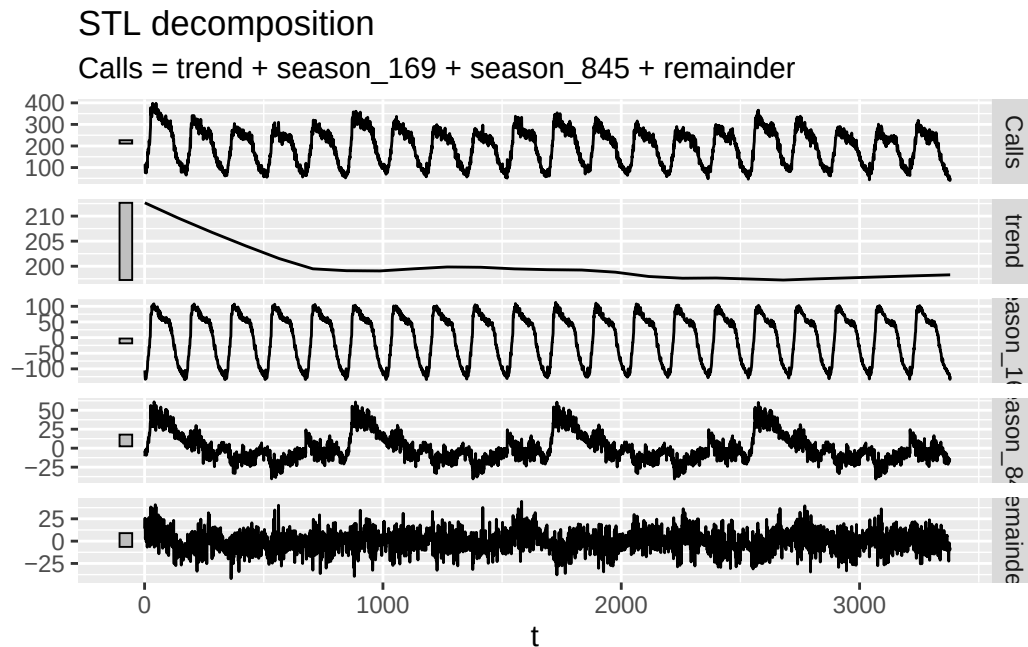
- c. Under this indexing, what are the periods of the seasonality?
- d. Apply STL decomposition with these seasonal periods. Which seasonality is stronger?
- e. Fit a dynamic harmonic regression model to this time series and compare its prediction performance with time series regression with seasonal dummies as well as DHR without an ARIMA terms.

```
bank_calls_filtered |>
  fill_gaps() |>
  autoplot(Calls)
```



We observe daily and weekly seasonality. The daily seasonality has period 169. The weekly seasonality has period $169 * 5 = 845$.

```
calls |>
  model(STL(Calls ~ season(period = 169) + season(period = 845))) |>
  components() |>
  autoplot()
```



Daily seasonality is stronger than weekly seasonality.

```
calls_fit <- calls |>
  model(dhr = ARIMA(Calls ~ PDQ(0, 0, 0) + pdq(d = 0) +
    fourier(period = 169, K = 10) +
    fourier(period = 5 * 169, K = 5)),
    seasonal_dummy = TSLM(Calls ~ season(5 * 169)),
    hr = TSLM(Calls ~ fourier(period = 5 * 169, K = 100)))
```

Warning: Provided exogenous regressors are rank deficient, removing regressors:
 `fourier(period = 5 \* 169, K = 5)C5\_845`, `fourier(period = 5 * 169, K =
 5)S5_845`

```
calls_test <- bank_calls |>
  mutate(t = row_number()) |>
  update_tsibble(index = t, regular = TRUE)
calls_fit |> forecast(h = 169 * 5) |> accuracy(calls_test)
```

```
# A tibble: 3 x 10
  .model      .type    ME  RMSE   MAE   MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 dhr        Test   4.11  22.3  17.1  1.44   9.88  1.12  1.12  0.559
2 hr         Test   3.14  21.2  16.2  0.846  9.50  1.06  1.07  0.520
3 seasonal_dummy Test   3.14  21.5  16.6  0.915  9.68  1.08  1.09  0.485
```

4. Electricity demand forecasting

Consider the `vic_elec` dataset, which measures the half-hourly electricity demand in Victoria, Australia, between 2012 and 2014. The time series has daily, weekly, and yearly seasonality. Furthermore, it also has two other covariates, measuring the daily temperature, and a dummy for public holidays. Let us use all this information to build a forecasting model.

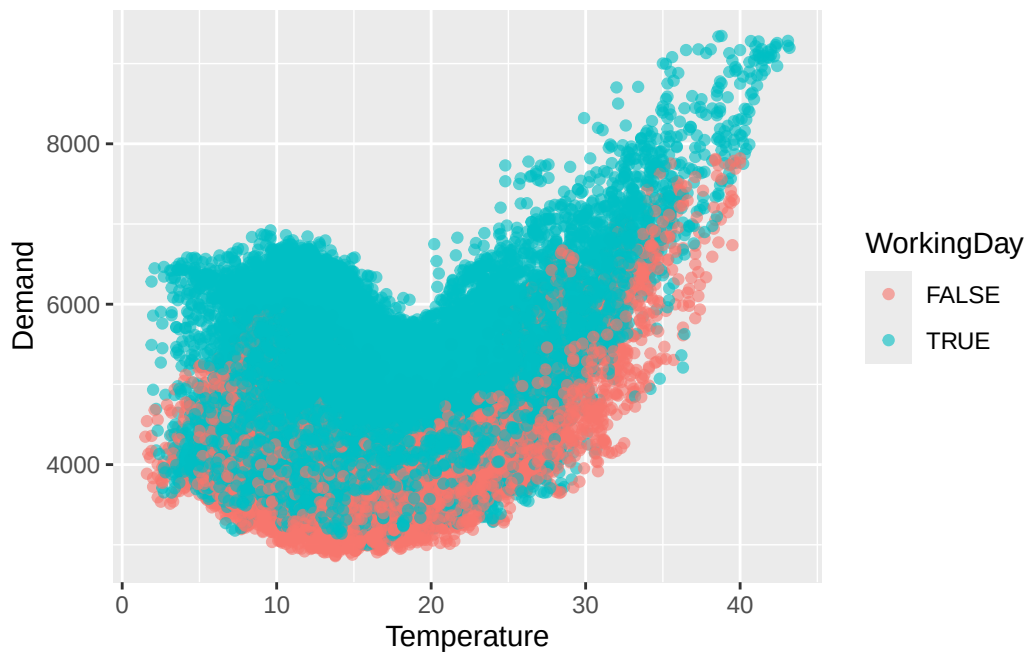
- Create a dummy for whether the day is a working day.
 - Make a scatter plot of Demand against Temperature, categorized according to whether the day is a working day. What do you observe?
 - Fit a dynamic harmonic regression model for Demand, using a quadratic function of the Temperature and the working day dummy as predictors.
 - Forecast the temperature for the next week.
-

```
vic_elec
```

```
# A tsibble: 52,608 x 5 [30m] <Australia/Melbourne>
  Time          Demand Temperature Date      Holiday
  <dtm>         <dbl>         <dbl> <date>    <lgl>
1 2012-01-01 00:00:00 4383.         21.4 2012-01-01 TRUE
2 2012-01-01 00:30:00 4263.         21.0 2012-01-01 TRUE
3 2012-01-01 01:00:00 4049.         20.7 2012-01-01 TRUE
4 2012-01-01 01:30:00 3878.         20.6 2012-01-01 TRUE
5 2012-01-01 02:00:00 4036.         20.4 2012-01-01 TRUE
6 2012-01-01 02:30:00 3866.         20.2 2012-01-01 TRUE
7 2012-01-01 03:00:00 3694.         20.1 2012-01-01 TRUE
8 2012-01-01 03:30:00 3562.         19.6 2012-01-01 TRUE
9 2012-01-01 04:00:00 3433.         19.1 2012-01-01 TRUE
```

```
10 2012-01-01 04:30:00 3359.          19.0 2012-01-01 TRUE
# i 52,598 more rows
```

```
elec <- vic_elec |>
  mutate(
    DOW = wday(Date, label = TRUE),
    WorkingDay = !Holiday & !(DOW %in% c("Sat", "Sun"))
  )
elec |>
  ggplot(aes(x = Temperature, y = Demand, color = WorkingDay)) + geom_point(alpha = 0.6)
```



Demand has a nonlinear relationship with Temperature and is also affected by whether the day is a working day.

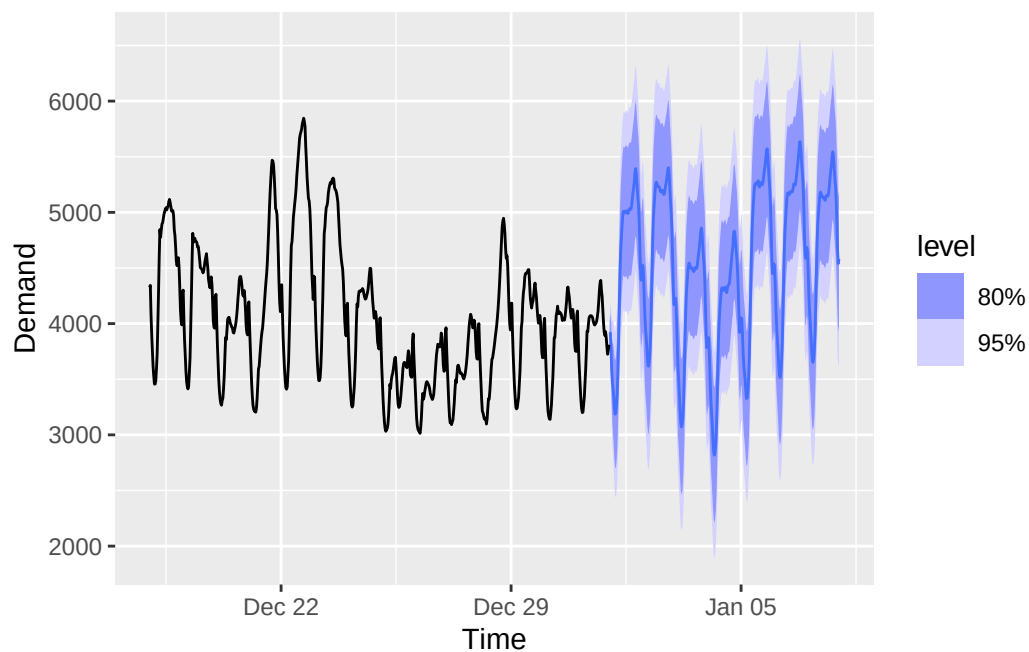
```
elec_fit <- elec |>
  model(ARIMA(Demand ~ WorkingDay + Temperature +
    I(Temperature ** 2) +
    fourier(period = "day", K = 10) +
    fourier(period = "week", K = 5) +
    fourier(period = "year", K = 3) +
    pdq(6, 0, 0) +
    PDQ(0, 0, 0) + 1))
```

```

elec_newdata <- new_data(elec, 7*48) |>
  mutate(
    Temperature = tail(elec$Temperature, 7 * 48),
    Date = lubridate::as_date(Time),
    DOW = wday(Date, label = TRUE),
    WorkingDay = (Date != "2015-01-01") &
      !(DOW %in% c("Sat", "Sun"))
  )
fc <- elec_fit |>
  forecast(new_data = elec_newdata)

fc |> autoplot(tail(elec, 48 * 14))

```



```

saveRDS(elec_fit, file = "elec_dhr_fit")

```